

Chia đôi, rồi lại chia đôi!

Xu Zhilei

2002

Nguồn gốc: Nhìn đa trọng chia đôi từ bài Mobiles (IOI 2001)

IOI China candidate team papers / 2002 / Xu Zhilei.pdf

1 Mở đầu

Chia đôi, với tư cách là một tư tưởng và phương pháp rất quan trọng, có vai trò khó thay thế trong nhiều phương diện. Tin học phát triển từng ngày, khiến tư tưởng “chia đôi” cũng nảy sinh những cách nhìn mới. Bài viết này muốn trình bày một vài nhận xét sơ lược của tác giả về một tư tưởng chia đôi khá mới: **đa trọng chia đôi**.

2 Bài toán

Giả sử khu vực Tampere được chia thành $N \times N$ ô, tạo thành một bảng $N \times N$. Tọa độ hàng và cột đều chạy từ 0 đến $N - 1$. Trong mỗi ô có một trạm phát tín hiệu điện thoại di động; các trạm này thỉnh thoảng báo cáo sự thay đổi của số điện thoại di động trong ô của mình. Hãy viết một chương trình nhận các báo cáo ấy và trả lời một số truy vấn: tổng số điện thoại di động hiện có trong một vùng hình chữ nhật nào đó.

Dữ liệu vào gồm nhiều dòng. Mỗi dòng chứa một số nguyên biểu thị loại thao tác, sau đó là một số nguyên làm tham số cho thao tác ấy, cụ thể như bảng sau.

Thao tác	Tham số	Ý nghĩa
0	N	Khởi tạo bảng kích thước $N \times N$, mọi ô đều có giá trị 0. Thao tác này chỉ xuất hiện một lần và chắc chắn là thao tác đầu tiên.
1	X, Y, A	Tăng số điện thoại di động trong ô (X, Y) thêm A . Giá trị A có thể dương hoặc âm.
2	L, T, R, B	Hỏi tổng số điện thoại di động trong hình chữ nhật từ góc trái trên (L, T) đến góc phải dưới (R, B) .
3	không có	Kết thúc chương trình. Thao tác này chỉ xuất hiện một lần và chắc chắn là thao tác cuối cùng.

Với mỗi thao tác loại 2, cần in ra một dòng chứa một số nguyên duy nhất, là đáp án của truy vấn tại thời điểm đó.

Các ràng buộc:

Kích thước bảng	$N \times N$	$1 \times 1 \leq N \times N \leq 1024 \times 1024$,
Giá trị của một ô tại mọi thời điểm	V	$0 \leq V \leq 2^{15} - 1 (= 32767)$,
Lượng tăng mỗi lần	A	$-2^{15} \leq A \leq 2^{15} - 1 (= 32767)$,
Tổng số thao tác trong dữ liệu vào	M	$3 \leq M \leq 60002$,
Tổng giá trị của mọi ô	S	$S \leq 2^{30}$.

3 Phân tích ban đầu

Gặp bài này, trước hết ta để nghĩ đến thuật toán đơn giản nhất: mở một mảng 1024×1024 , ghi số điện thoại di động trong từng ô, ban đầu đặt tất cả bằng 0. Sau đó đọc lần lượt các lệnh và thực hiện. Thao tác “sửa đổi” chỉ cần thay đổi trực tiếp ô tương ứng trong mảng; thao tác “truy vấn” thì phải xét mọi ô từ góc trái trên đến góc phải dưới, cộng số điện thoại di động trong các ô ấy rồi in ra.

Ưu điểm của thuật toán này là đơn giản, tự nhiên và dễ cài đặt. Nhược điểm là độ phức tạp thời gian quá cao. Mỗi thao tác “sửa đổi” mất thời gian hằng số, nhưng mỗi thao tác “truy vấn” có thể lên tới cấp $\mathcal{O}(n^2)$. Trong trường hợp xấu nhất, độ phức tạp là

$$\mathcal{O}(mn^2) = 60000 \cdot 1024 \cdot 1024,$$

rõ ràng không thể cho kết quả trong thời gian quy định.

4 Phương pháp chia đôi

Để tiện phân tích, ta đơn giản hóa bài toán bằng cách hạ xuống một chiều: chỉ xét một hàng trong bảng. Thao tác “sửa đổi” thay đổi số điện thoại di động trong một ô của hàng này, còn thao tác “truy vấn” hỏi từ ô L đến ô R có tổng cộng bao nhiêu điện thoại di động.

Nếu dùng thuật toán đơn giản nói trên, thao tác “sửa đổi” có độ phức tạp $\mathcal{O}(1)$, thao tác “truy vấn” có độ phức tạp $\mathcal{O}(n)$, và tổng thể là $\mathcal{O}(mn)$, rất kém.

Để cải tiến, ta tự nhiên thu được một tư tưởng chia đôi: chia n ô từ 1 đến n thành hai đoạn, rồi tiếp tục chia đôi từng đoạn, đồng thời ghi lại tổng số điện thoại di động của mỗi đoạn. Cách làm này tạo thành một cấu trúc dạng cây.

Ví dụ, với một hàng gồm tám ô như sau:

Tọa độ	0	1	2	3	4	5	6	7
Số điện thoại	3	5	2	1	4	6	1	3

tổng theo các đoạn của cây chia đôi là:

Mức	Các đoạn và tổng tương ứng
0	[0, 7] (25)
1	[0, 3] (11), [4, 7] (14)
2	[0, 1] (8), [2, 3] (3), [4, 5] (10), [6, 7] (4)
3	[0, 0] (3), [1, 1] (5), [2, 2] (2), [3, 3] (1), [4, 4] (4), [5, 5] (6), [6, 6] (1), [7, 7] (3)

Đây là bản dựng lại bằng bảng của hình cây trong bản gốc; số trong ngoặc là tổng số điện thoại trên đoạn.

Sau khi chia đôi, thao tác “sửa đổi” cần sửa các đoạn nằm trên đường đi từ “gốc” tới “lá”, nên độ phức tạp là $\mathcal{O}(\log_2 n)$. Vậy thao tác “truy vấn” thực hiện thế nào?

Chú ý rằng tổng số điện thoại trong đoạn $[L, R]$ bằng tổng của đoạn $[0, R]$ trừ đi tổng của đoạn $[0, L - 1]$. Tổng của đoạn $[0, X]$ có thể tính như sau. Ban đầu đặt $Total = 0$. Từ “gốc” đi xuống để tìm X . Nếu ở một bước nào đó ta đi sang cây con phải, thì cộng toàn bộ số điện thoại của cây con trái vào $Total$. Cuối cùng khi tìm tới X , cộng thêm số ghi tại nút “lá”. Khi đó $Total$ chính là tổng số điện thoại trong đoạn $[0, X]$.

Như vậy thao tác “truy vấn” cũng có độ phức tạp $\mathcal{O}(\log_2 n)$, và toàn bộ thuật toán chỉ còn $\mathcal{O}(m \log_2 n)$, rất tốt.

5 Đa trọng chia đôi

Phần trên phân tích trường hợp một chiều và, dựa trên tư tưởng chia đôi, thu được một thuật toán khá tốt. Thuật toán ấy có thể mở rộng sang trường hợp hai chiều hay không? Câu trả lời là có.

Hãy suy nghĩ một chút: trong trường hợp một chiều, đối tượng của “chia đôi” là “đoạn”, hoặc có thể xem là “đường”, và tiêu chuẩn để chia là tọa độ X . Khi mở rộng lên hai chiều, đối tượng của “chia đôi” tất nhiên phải là “mặt”, tức bảng; tiêu chuẩn để chia cần xét đồng thời tọa độ X và Y . Lúc này “chia đôi” liên quan tới hai tiêu chuẩn khác nhau, nên ta gọi nó là **đa trọng chia đôi**.

Chia như thế nào? Trước hết, theo tọa độ X , ta chia toàn bộ bảng thành các phần, rồi tiếp tục chia đôi từng phần theo tọa độ Y . Đồng thời, mỗi phần thu được lại được chia đôi theo tọa độ Y , và ta ghi lại tổng số điện thoại di động của từng phần cuối cùng.

Ví dụ, bảng 2×2 sau:

$Y \backslash X$	0	1
0	3	2
1	1	0

được biểu diễn bằng cây hai tầng như sau. Ở tầng ngoài, ta chia theo X ; tại mỗi nút của tầng ngoài lại có một cây chia theo Y :

Nút theo X	Các nút theo Y và tổng
$X : [0, 1]$	$Y : [0, 1](6), \quad Y : [0, 0](5), \quad Y : [1, 1](1)$
$X : [0, 0]$	$Y : [0, 1](4), \quad Y : [0, 0](3), \quad Y : [1, 1](1)$
$X : [1, 1]$	$Y : [0, 1](2), \quad Y : [0, 0](2), \quad Y : [1, 1](0)$

Kết quả của “đa trọng chia đôi” thực ra tương tự kết quả chia đôi trong trường hợp một chiều: nó cũng tạo thành một cấu trúc dạng “cây”. Điểm khác là do đây là chia đôi nhiều tầng, mỗi nút của “cây” này lại vẫn là một “cây”.

Trên “cây chia đôi” ấy, để thực hiện thao tác “sửa đổi”, ta chỉ cần theo tọa độ X đi từ “gốc” tới “lá” và xử lý mọi nút trên đường đi. Vì mỗi nút này lại là một cây, trong từng nút ta tiếp tục theo tọa độ Y đi từ “gốc” tới “lá”, lần lượt sửa dữ liệu trên đường đi. Như vậy, xử lý mỗi nút mất $\mathcal{O}(\log_2 n)$, số nút cần xử lý trong một lần sửa là $\log_2 n$, nên mỗi thao tác “sửa đổi” có độ phức tạp

$$\mathcal{O}(\log_2 n \cdot \log_2 n).$$

Thao tác “truy vấn” có thể bắt chước thuật toán một chiều. Trước hết, tổng số điện thoại trong hình chữ nhật từ góc trái trên (L, T) tới góc phải dưới (R, B) bằng tổng trong hình chữ nhật từ $(0, 0)$ tới (R, B) , trừ tổng từ $(0, 0)$ tới $(L - 1, B)$, trừ tổng từ $(0, 0)$ tới $(R, T - 1)$, rồi cộng lại tổng từ $(0, 0)$ tới $(L - 1, T - 1)$:

$$\text{sum}(L, T, R, B) = F(R, B) - F(L - 1, B) - F(R, T - 1) + F(L - 1, T - 1),$$

trong đó $F(X, Y)$ là tổng trên hình chữ nhật từ $(0, 0)$ đến (X, Y) . Hình minh họa trong bản gốc chính là cách tách bốn vùng theo công thức bao hàm–loại trừ này.

Tổng số điện thoại trong hình chữ nhật từ $(0, 0)$ đến (X, Y) cũng có thể tính theo cách tương tự trường hợp một chiều. Ban đầu đặt $Total = 0$. Trên “cây chia đôi”, từ “gốc” đi tới “lá” để tìm X . Nếu ở một bước nào đó đi sang cây con phải, ta cần xử lý nút gốc của cây con trái tương ứng; sau cùng khi tìm tới X , xử lý cả nút lá ấy.

Mỗi nút trên “cây chia đôi” theo tọa độ X đồng thời là một “cây chia đôi” theo tọa độ Y . Cách xử lý nó là: từ “gốc” đi tới “lá” để tìm Y . Nếu ở một bước nào đó đi sang cây con phải, thì cộng tổng số điện thoại của toàn bộ phần thuộc cây con trái tương ứng vào $Total$. Cuối cùng khi tìm tới Y , cộng giá trị tại nút lá.

Sau quá trình này, giá trị $Total$ thu được chính là tổng số điện thoại trong hình chữ nhật từ $(0, 0)$ đến (X, Y) . Trên “cây chia đôi” theo tọa độ X , xử lý mỗi nút mất $\mathcal{O}(\log_2 n)$; mỗi lần thống kê như vậy cần xử lý $\log_2 n$ nút loại này; mỗi thao tác “truy vấn” lại cần thực hiện bốn lần thống kê. Vì thế độ phức tạp của một thao tác “truy vấn” là

$$\mathcal{O}(\log_2 n \cdot \log_2 n).$$

Do đó độ phức tạp của toàn bộ thuật toán là

$$\mathcal{O}(m \log_2 n \log_2 n),$$

rất nhanh và đủ để giải trong giới hạn thời gian.

6 Tổng kết

Qua việc thảo luận lời giải bài Mobiles, ta có nhận thức sâu hơn về tư tưởng và phương pháp “chia đôi”. Nói chung, phương pháp “chia đôi” thường tạo ra một “cây chia đôi”. Nếu tiêu chuẩn để “chia” chỉ có một, thì nút của cây chia đôi là dữ liệu liên quan trực tiếp tới đối tượng cần xử lý. Khi đối tượng cần xử lý liên quan tới nhiều tiêu chuẩn khác nhau, ta thường có thể dùng “đa trọng chia đôi”; khi ấy, nút của cây chia đôi nên là một cây chia đôi cấp dưới. Có thể hình dung rằng nếu đối tượng liên quan tới Z tiêu chuẩn chia đôi khác nhau, thì sẽ có Z cấp cây chia đôi, và độ phức tạp khi thao tác trên đối tượng là

$$\mathcal{O}((\log_2 n)^Z).$$

Thuật toán rút ra từ tư tưởng “đa trọng chia đôi” vì vậy thường là thuật toán rất tốt.

Có thể thấy, khi bài toán trước mắt thích hợp để giải bằng chia đôi, mà tiêu chuẩn “chia” lại có nhiều hơn một, ta nên nghĩ tới đa trọng chia đôi. Nó là sản phẩm kết hợp của hai tư tưởng quan trọng: tư tưởng “chia đôi” và tư tưởng “hạ chiều”, có thể giảm độ phức tạp thời gian một cách hiệu quả. Đừng do dự nữa: chia đôi, rồi lại chia đôi!

A Phụ lục: đề gốc Mobiles

Đề gốc Mobiles như sau. Trong bài viết, để tiện thảo luận, phần dịch ở trên có một vài thay đổi nhỏ.

Mobile phones

Bài toán. Giả sử các trạm gốc điện thoại di động thể hệ thứ tư trong khu vực Tampere hoạt động như sau. Khu vực được chia thành các ô vuông, tạo thành một ma trận $S \times S$, với hàng và cột được đánh số từ 0 đến $S - 1$. Mỗi ô chứa một trạm gốc. Số điện thoại di động đang hoạt động trong một ô có thể thay đổi vì điện thoại được di chuyển từ ô này sang ô khác, hoặc được bật hay tắt. Thỉnh thoảng, mỗi trạm gốc báo cáo sự thay đổi số điện thoại đang hoạt động cho trạm chính, kèm theo hàng và cột của ô trong ma trận.

Hãy viết một chương trình nhận các báo cáo này và trả lời truy vấn về tổng số điện thoại di động đang hoạt động trong bất kỳ vùng hình chữ nhật nào.

Vào và ra. Dữ liệu vào được đọc từ chuẩn vào dưới dạng các số nguyên; đáp án cho các truy vấn được ghi ra chuẩn ra dưới dạng các số nguyên. Dữ liệu vào được mã hóa như sau. Mỗi dòng là một lệnh riêng, gồm một số nguyên chỉ lệnh và một số số nguyên tham số theo bảng sau.

Lệnh	Tham số	Ý nghĩa
0	S	Khởi tạo ma trận kích thước $S \times S$, mọi ô đều bằng 0. Lệnh này chỉ được cho một lần và là lệnh đầu tiên.
1	X, Y, A	Cộng A vào số điện thoại đang hoạt động trong ô (X, Y) . A có thể dương hoặc âm.
2	L, B, R, T	Hỏi tổng hiện tại của số điện thoại di động đang hoạt động trong các ô (X, Y) thỏa $L \leq X \leq R$ và $B \leq Y \leq T$.
3	không có	Kết thúc chương trình. Lệnh này chỉ được cho một lần và là lệnh cuối cùng.

Các giá trị luôn nằm trong phạm vi hợp lệ, nên không cần kiểm tra. Đặc biệt, nếu A âm, có thể giả sử rằng nó không làm giá trị của ô giảm xuống dưới không. Chỉ số bắt đầu từ 0; chẳng hạn với bảng kích thước 4×4 , ta có $0 \leq X \leq 3$ và $0 \leq Y \leq 3$.

Chương trình không được in gì đối với các dòng có lệnh khác 2. Nếu lệnh là 2, chương trình cần trả lời truy vấn bằng cách ghi đáp án trên một dòng duy nhất, chứa một số nguyên duy nhất, ra chuẩn ra.

Hướng dẫn lập trình. Trong các ví dụ dưới đây, số nguyên `last` là số cuối cùng được đọc từ một dòng, còn `answer` là biến số nguyên chứa đáp án.

Nếu lập trình bằng C++ và dùng `iostream`, nên dùng cách đọc chuẩn vào và ghi chuẩn ra sau:

```
cin>>last;
cout<<answer<<endl<<flush;
```

Nếu lập trình bằng C hoặc C++ và dùng `scanf/printf`, nên dùng:

```
scanf ("%d", &last);
printf("%d\n",answer); fflush (stdout);
```

Nếu lập trình bằng Pascal, nên dùng cách đọc chuẩn vào và ghi chuẩn ra sau:

```
Read(last); ... Readln;
Writeln(answer);
```

Ví dụ.

stdin	giải thích
0 4	Khởi tạo bảng kích thước 4×4 .
1 1 2 3	Cập nhật ô $(1, 2)$ thêm 3.
2 0 0 2 2	Hỏi tổng trong $0 \leq X \leq 2, 0 \leq Y \leq 2$.
1 1 1 2	Cập nhật ô $(1, 1)$ thêm 2.
1 1 2 -1	Cập nhật ô $(1, 2)$ bớt 1.
2 1 1 2 3	Hỏi tổng trong $1 \leq X \leq 2, 1 \leq Y \leq 3$.
3	Kết thúc chương trình.

Các dòng đáp án tương ứng trên chuẩn ra là:

```
3
4
```

Ràng buộc.

Kích thước bảng	$S \times S$	$1 \times 1 \leq S \times S \leq 1024 \times 1024$,
Giá trị ô tại mọi thời điểm	V	$0 \leq V \leq 2^{15} - 1 (= 32767)$,
Lượng cập nhật	A	$-2^{15} \leq A \leq 2^{15} - 1 (= 32767)$,
Số lệnh trong dữ liệu vào	U	$3 \leq U \leq 60002$,
Số điện thoại tối đa trong toàn bảng	M	$M = 2^{30}$.

Trong 20 bộ dữ liệu, có 16 bộ có kích thước bảng không quá 512×512 . Ghi chú: hệ thống kiểm tra trên web đưa tệp dữ liệu vào tới chuẩn vào của chương trình.

B Phụ lục: chương trình Mobiles.pas

Chương trình Pascal trong bản gốc:

```
var
  index      :array[0..1023,0..15] of integer;
  s          :array[0..1023,0..1023] of longint;
  n,cmd      :integer;

procedure init;
var
  m,low,up,mid :integer;
begin
  read(m);
  n:=1;
  while n<m do n:=n shl 1;
  dec(n);
  fillchar(index,sizeof(index),0);
  for m:=0 to n do
  begin
    low:=0;up:=n;
    while low<up do
    begin
      if up=m then break;
      mid:=(low+up) shr 1;
      if mid<m then begin inc(index[m,0]);
                        index[m,index[m,0]]:=mid;low:=mid+1;
                        end
                      else begin inc(index[m,0]);
                        index[m,index[m,0]]:=up;up:=mid;
                        end;
    end;
    inc(index[m,0]);
    index[m,index[m,0]]:=m;
  end;
end;
```

```

procedure update;
var
  x,y,d,i,j      :integer;
begin
  read(x,y,d);
  for i:=1 to index[x,0] do
    if index[x,i]>=x then
      for j:=1 to index[y,0] do
        if index[y,j]>=y then inc(s[index[x,i],index[y,j]],d);
      end;
    end;
  end;

function rect(x,y:integer):longint;
var
  i,j      :integer;
  total    :longint;
begin
  if (x<0) or (y<0) then begin rect:=0;exit;
                        end;

  total:=0;
  for i:=1 to index[x,0] do
    if index[x,i]<=x then
      for j:=1 to index[y,0] do
        if index[y,j]<=y then inc(total,s[index[x,i],index[y,j]]);
      end;
    rect:=total;
  end;

procedure query;
var
  left,bottom,right,top:integer;
begin
  read(left,top,right,bottom);
  writeln(rect(right,bottom)-rect(left-1,bottom)
          -rect(right,top-1)+rect(left-1,top-1));
end;

BEGIN
  repeat
    read(cmd);
    case cmd of
      0:init;
      1:update;
      2:query;
    end;
  until cmd=3;
END.

```